

SAP Self-Healing System

Real-World Integration Guide

This guide covers how to deploy, configure, and extend the SAP Self-Healing project in a real SAP RISE / SAP Cloud ALM environment — from adding runbooks to connecting ServiceNow and SAP Event Mesh.

Project	github.com/Sai6522/heal
Stack	Python 3.10 · FastAPI · LangChain · ChromaDB · Ollama / OpenAI
Target	SAP RISE · SAP Cloud ALM · SAP BTP · ServiceNow

Step 1 — Add Real Runbooks

Every SAP error you want to auto-heal needs a YAML runbook in the **runbooks/** folder. The LLM uses the runbook content to validate its diagnosis and assign a confidence score. Higher quality runbooks = higher confidence = more auto-closures.

Runbook Template

```
# runbooks/YOUR_ERROR_CODE.yaml
error_code: YOUR_ERROR_CODE          # must match exactly what SAP ALM sends
title: Human readable title
category: ABAP|RFC|BATCH|DB|IDOC|INTERFACE

symptoms:
  - What you see in the system (transaction codes, error messages)

root_cause: >
  Explain WHY this happens. The LLM uses this to validate its diagnosis.
  More detail = higher confidence scores.

remediation_steps:
  - "1. Transaction -> what to do"
  - "2. Next step"
  - "3. Verification step"

auto_close_eligible: true             # false = always needs human (security issues etc.)

references:
  - "SAP Note 1234567"
```

Recommended Error Codes to Add

Error Code	SAP Area	Key Transactions
DBIF_RSQSQL_ERROR	DB lock / timeout	ST22, DB02
TSV_TNEW_PAGE_ALLOC_FAILED	Memory overflow	ST22, SM50

MESSAGE_TYPE_X	ABAP runtime error	ST22
ENQUEUE_FAILED	Lock table overflow	SM12, SM21
SPOOL_NO_INFO	Spool overflow	SP01, SP12
IDOC_STATUS_FAILED	IDoc status 02/56	WE02, BD87
SM59_RFC_ERROR	RFC config broken	SM59
BATCH_JOB_ABORTED	Job aborted	SM37, SM36

After adding any YAML file, re-index ChromaDB:

```
python main.py index
```

Step 2 — Connect Real SAP Cloud ALM

Get these values from **SAP BTP Cockpit** → **your subaccount** → **SAP Cloud ALM** → **Service Keys**.

```
# .env
SAP_ALM_BASE_URL=https://YOUR-TENANT.alm.cloud.sap
SAP_ALM_CLIENT_ID=sb-your-actual-client-id
SAP_ALM_CLIENT_SECRET=your-actual-secret
SAP_ALM_TOKEN_URL=https://YOUR-TENANT.authentication.eu10.hana.ondemand.com/oauth/token
```

Navigation Path in BTP Cockpit

- BTP Cockpit → Subaccount → Instances and Subscriptions
- SAP Cloud ALM → Service Keys → Create Key
- Copy: clientid, clientsecret, url

Test the Connection

```
python -c "from sap.alm_client import get_open_incidents; print(get_open_incidents())"
```

Step 3 — Set Up Webhook (Real-time Push)

The webhook replaces polling — SAP Cloud ALM calls your app the moment an alert fires. This gives you near-instant response instead of waiting for the next poll cycle.

1. Deploy to SAP BTP Cloud Foundry

```
# manifest.yml (create in project root)
applications:
- name: sap-healing
  memory: 512M
  command: python main.py api
  buildpacks: [python_buildpack]
```

```
cf push sap-healing
```

2. Register Webhook in SAP Cloud ALM

- SAP Cloud ALM → Operations → Alert Notification
- Create Notification → Type: Webhook
- URL: <https://sap-healing.cfapps.eu10.hana.ondemand.com/events/webhook/alm-alert>

- Secret: same value as WEBHOOK_SECRET in your .env
- Events: Alert Created, Alert Updated

3. Test with curl

```
curl -X POST https://your-app/events/webhook/alm-alert/demo \
-H "Content-Type: application/json" \
-d '{"alertId":"INC-TEST","alertTitle":"ABAP dump",
    "errorCode":"SYSTEM_NO_ROLL","severity":"high","systemId":"PRD"}'
```

Step 4 — Connect Real ServiceNow

The system creates a ServiceNow ticket the moment an alert arrives, posts the AI-generated RCA as a work note, then closes or assigns the ticket based on confidence.

1. Get a Free Dev Instance

Go to **developer.servicenow.com** → Request Instance (free, takes ~5 min)

2. Set Credentials in .env

```
SNOW_INSTANCE=dev12345.service-now.com
SNOW_USER=admin
SNOW_PASS=your-actual-password
SNOW_ASSIGNMENT_GROUP=SAP-BASIS-TEAM
```

3. Create Assignment Group in ServiceNow

- ServiceNow → User Administration → Groups → New
- Name: SAP-BASIS-TEAM (must match SNOW_ASSIGNMENT_GROUP exactly)
- Add your SAP Basis team members

4. Test

```
python -c "
from integrations.servicenow import create_snow_incident
sid = create_snow_incident({'id':'TEST-001','title':'Test',
    'error_code':'TEST','priority':'low','system_id':'DEV'})
print('Created SNOW ticket:', sid)
"
```

ServiceNow Ticket Lifecycle

Event	ServiceNow Action
Alert received	Incident created (state: New)
LLM RCA complete	Work note added with root cause + steps + confidence %
confidence >= threshold	Incident resolved (Solved by AI)
confidence < threshold	Incident assigned to SAP-BASIS-TEAM for manual review

Step 5 — Map SAP Cloud ALM Automation Runbooks

For each error code, create a pre-built automation in SAP Cloud ALM. The system triggers these automations to execute the actual fix inside SAP.

Create Automation in SAP Cloud ALM

- SAP Cloud ALM → Intelligent Event Processing → Automation → Create
- Name: rb-system-no-roll-fix
- Add steps: restart work process, increase em/initial_size_MB
- Save → copy the Automation ID

Map IDs in .env

```
ALM_RUNBOOK_SYSTEM_NO_ROLL=<actual-automation-id>
ALM_RUNBOOK_RFC_TIMEOUT=<actual-automation-id>
ALM_RUNBOOK_JOB_FAILED=<actual-automation-id>
ALM_RUNBOOK_IDOC_ERROR=<actual-automation-id>
ALM_RUNBOOK_INTERFACE_TIMEOUT=<actual-automation-id>
```

Step 6 — Tune Policy Guardrails

The policy engine decides whether auto-remediation is allowed before any action is taken. Tune these settings to match your company's change management rules.

```
# .env
PRD_SYSTEMS=PRD,P01,P1          # your actual production SIDs
MIN_CONFIDENCE_PRD=0.90         # stricter threshold for production
MIN_CONFIDENCE_DEV=0.75         # threshold for dev/QA systems
MANUAL_ONLY_CODES=AUTH_FAILURE,SECURITY_ALERT # never auto-close these
CHANGE_FREEZE=false             # set true during SAP upgrades/maintenance
BUSINESS_HOURS_ONLY=true        # only auto-close 8am-6pm
BUSINESS_HOURS_START=8
BUSINESS_HOURS_END=18
```

Guardrail	Behaviour
CRITICAL priority	Always blocked — escalate to on-call
PRD system	Requires MIN_CONFIDENCE_PRD (default 90%)
CHANGE_FREEZE=true	All auto-remediation blocked
Failed 2+ times	Escalates to SAP Basis team instead of retrying
MANUAL_ONLY_CODES	Listed error codes always go to human review
BUSINESS_HOURS_ONLY	Queues actions outside configured hours

Step 7 — Run in Production

Start the API Server (always-on, receives webhooks)

```
python main.py api
```

Run One Healing Cycle Manually

```
python main.py heal
```

Re-index After Adding New Runbooks

```
python main.py index
```

Recommended Production Setup on SAP BTP

Component	Command	Purpose
API Server	python main.py api	Always-on, receives webhook alerts instantly
Cron (15 min)	python main.py heal	Catches anything webhooks missed (fallback poll)
On-demand	python main.py index	Run after adding/updating runbooks

Runbook Quality Tips

The LLM confidence score is directly tied to runbook quality. Better runbooks = higher confidence = more incidents auto-closed without human intervention.

Tip	Why It Matters
Use exact transaction codes (ST22, SM50, WE02)	LLM recognises SAP transactions and scores them as authoritative
Include the error_code field exactly as SAP sends it	Triggers the 0.85 confidence floor — biggest single confidence boost
Add SAP Note numbers in references	LLM treats SAP Notes as official documentation
Set auto_close_eligible: false for security/auth errors	Prevents accidental auto-closure of sensitive incidents
Start new runbooks with auto_close_eligible: false	Watch RCA quality for a week, then flip to true once you trust it
Add real symptoms from your system logs	Improves RAG retrieval — more specific = better runbook match